

Pro_AmineUMT IDE with AI — Architecture Supplement

v4.2 — UMT Debug Model: Source-Level Debugging

"Debug at Source, Run at Native"

© Pro_Amine LLC (United States) · Developed by Amine Saoud ibn al-Bashir · Technology Innovator Awards 2025 — Rising Star in Microchip Technology Solutions in Europe

A. The Architectural Question

When a language uses Pure VR Source / Hidden Native Backend, debugging exposes a fundamental design question: should the debugger operate at the **source level** (UMT VR API) or at the **generated level** (platform-specific C++)? This has been answered consistently by every successful high-level language.

B. How Successful Languages Solve It

Language	Source	Compiled	Debugger sees	Mechanism
TypeScript	.ts	JavaScript	.ts	Source maps (.js.map)
CUDA	.cu	PTX/SASS	.cu	DWARF debug info
Dart / Flutter	.dart	native ARM	.dart	Dart source map
Kotlin	.kt	JVM bytecode	.kt	JVMTI + debug attrs
UMT	.umt	platform C++	.umt	aia_map.json + ELF

C. The UMT Decision — Debug at Source, Run at Native

UMT debug operates at the **.umt source level**, NOT at the generated C++ level. This is the **second foundational architectural decision** of UMT Platform, after Pure VR Source / Hidden Native Backend.

Debugging at the generated C++ level would break four of the five Pure VR Source Guarantees. UMT debug must mirror TypeScript: breakpoints, variables, and stack traces all live at the source level.

D. Mechanism — The aia_map.json Source Map

The Abstraction Intelligence Algorithm Engine emits a source map alongside the generated C++, at **.umt/cache/aia_map.json**. It contains:

- Bidirectional line mappings: umt-line ↔ cpp-line
- VR-to-physical-GPIO resolution per target
- Variable-name remapping (VR-level vs framework-level)
- Method-call to framework-call translations

D.1 — Example source map for the VR_ADC4 project

```
{
  "version": 1,
  "source": "adc_read.umt",
  "generated": ".umt/generated/main_generated.cpp",
  "target": "nanokit-esp32",
  "toolchain": "arduino-esp32",
  "build_mode": "debug",
  "mappings": [
```

```
    { "umt": 2, "cpp": 4,  "vr": "VR_ADC4",    "gpio": 32, "method": "activate" },
    { "umt": 3, "cpp": 5,  "vr": "VR_ADC_IN4", "gpio": 32, "method": "enable" },
    { "umt": 7, "cpp": 10, "vr": "VR_ADC_IN4", "gpio": 32, "method": "read" }
  ],
  "vr_resolution": {
    "VR_ADC4":    { "peripheral": "ADC1", "channel": "CH4" },
    "VR_ADC_IN4": { "pin": "GPIO32", "peripheral": "ADC1_CH4" }
  },
  "variable_views": {
    "VR_ADC_IN4.value": { "cpp": "analogRead_result", "format": "int_0_4095" },
    "VR_ADC_IN4.state": { "cpp": "_umt_state[VR_ADC_IN4]", "format": "enum" }
  }
}
```

E. The Debug Flow

When a developer sets a breakpoint and runs in debug mode:

- Developer sets breakpoint in **adc_read.umt** line 7 (VR API call)
- IDE reads aia_map.json → finds { umt: 7, cpp: 10 }
- IDE issues GDB command: "set breakpoint at main_generated.cpp:10"
- GDB reads ELF debug symbols → locates 0x400D2A4F as machine address for cpp:10
- GDB programs hardware breakpoint via JTAG/SWD at 0x400D2A4F
- ESP32 executes, reaches 0x400D2A4F, halts
- GDB reports: "stopped at 0x400D2A4F"
- IDE reverse-maps: 0x400D2A4F → cpp:10 → umt:7
- IDE highlights **adc_read.umt** line 7 (the VR call the developer wrote)
- Variables panel shows VR-level view, never framework-level details

F. Variable Inspection — VR-Level vs Framework-Level

When a breakpoint hits inside **UMT.Interface_Pin(VR_ADC_IN4).read()**, the developer sees:

Shown to developer (VR-level)	Hidden (framework-level)
value = 2048	(local — same in both views)
VR_ADC_IN4.value = 2048	adc1_get_raw(ADC1_CHANNEL_4) = 0x800
VR_ADC_IN4.state = ENABLED	_umt_state[VR_ADC_IN4] = 2
VR_ADC_IN4.peripheral = ADC1_CH4	(implementation detail)
VR_ADC_IN4.resolution = 12 bits	adc1_config_width(ADC_WIDTH_BIT_12)

G. Optional Advanced View

Following CUDA's model, UMT offers a read-only side panel showing the generated C++:

- Hidden by default — must be explicitly enabled
- Red watermark: **AIA-GENERATED · DO NOT EDIT · WIPED ON BUILD** ⚠
- Breakpoints cannot be set in this view — they remain in .umt
- 99% of UMT developers never open this view

H. Run vs Debug Mode

Aspect	Run Mode	Debug Mode
Compile flags	Release (-O2)	Debug (-g, -O0)
Flash method	UART (esptool)	JTAG/SWD probe
Breakpoints	Disabled	Hardware breakpoints active
Variables panel	N/A	VR-level + Watch + Locals
Source maps	Optional	Always generated
Performance	Maximum (production)	Reduced (instrumentation)
Probe required	No	Yes (per-target probe)

I. Why This Decision Is Foundational

Debug at Source, Run at Native is the second foundational decision of UMT Platform. Together with Pure VR Source, they form a closed system: the abstraction is never broken — not at writing, not at building, not at running, not at debugging. The Hidden Native Backend remains hidden across the entire development lifecycle.